

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра математического обеспечения и применения ЭВМ

ОТЧЕТ
по практической работе №1
по дисциплине «Системы параллельной обработки данных»
Тема: Использование функции обмена данными “Точка-Точка” в
библиотеке MPI

Студент гр. 1306

Шаврин А.П.

Преподаватель

Татаринов Ю.С.

Санкт-Петербург

2025

Цель работы.

Изучить, как использовать функцию обмена данными «точка-точка» в библиотеке MPI. Реализовать программу, использующую данную функцию как основную часть для решения задачи.

Постановка задачи.

Вариант 10. Морской бой. Несколько процессов поочередно обмениваются сообщениями согласно правилам игры в морской бой. (правила игры с числом игроков больше двух – устанавливаются самостоятельно)

Выполнение работы.

В данной реализации многопользовательского морского боя участвуют несколько процессов, организованные в кольцевую топологию, где каждый игрок последовательно обменивается сообщениями по следующим правилам: игроки размещают на своих полях однопалубные корабли согласно балансировочной формуле $COUNT_SHIPS = (FIELD_SIZE^2 \times DENSITY_PERCENT) / 100$, где плотность $DENSITY_PERCENT = 20\%$ соответствует стандартному морскому бою. Затем в каждом раунде происходит синхронизация состояний всех участников, после чего каждый живой игрок атакует ближайшего живого противника по часовой стрелке в кольце, отправляя координаты выстрела, и одновременно защищается от атаки ближайшего живого игрока против часовой стрелки, обрабатывая входящий выстрел и возвращая результат. Игра продолжается до тех пор, пока не останется только один выживший игрок, который объявляется победителем.

Сначала были определены макроподстановки с помощью директивы `#define` для размера поля, процента клеток под корабли от общего размера поля и количества кораблей, рассчитываемое из предыдущих 2х директив. Затем при помощи перечислений были определены состояния игрока/процесса (жив/убит), состояния клетки (вода/корабль) и результата выстрела (мимо/попал/убил). Каждая клетка описана при помощи структуры, хранящей свои координаты и состояние.

Для каждого процесса создается собственное игровое поле – одномерный вектор клеток, описывающий двумерное поле. Корабли размещаются так, что один корабль не может стоять там, где стоит другой, но они могут стоять в соседних клетках. Также для каждого процесса создается остаточная карта клеток противника, чтобы игроки избегали тех клеток, по которым выстрелы уже совершались. Процесс выбывает из игры, если количество его кораблей становится равным нулю. Игра заканчивается, когда остается один процесс.

Определенные два статуса игроков (живой и мертвый) заданы по следующему правилу: Живой игрок может быть атакующим и защищающимся, то есть осуществлять выбор противника, по которому будет стрелять и от кого получать урон, выбор клетки для атаки, выстрелы по сопернику, прием удара от другого соперника и получение результата своей атаки. Мертвый игрок ждет окончания игры, когда он может перейти в состояние проигравшего, его задача заключается в уведомлении в начале каждого хода остальных игроков о своем статусе.

Обмен состояниями игроков происходит в начале каждого хода при помощи коллективной функции `MPI_Allgather`, что бы каждый игрок мог получить состояния всех игроков и определить, в какое состояние ему переходить: победитель, проигравший, наблюдатель или активный игрок. Игрок считается победителем, если остался жив, только он один. Проигравшим считается процесс, если остался жив только один игрок и это не он. Наблюдателем процесс становится если его статус – мертв и осталось не менее 2х живых игроков. Активным игроком процесс становится, если он жив и осталось не менее 2х живых игроков.

Для визуального представления алгоритма нарисована сеть Петри для 3х игроков представленная на рисунке 1. Начальными для каждого процесса являются состояния ожидания следующего хода, конечными являются

состояния победителя и проигравшего, также для каждого процесса.

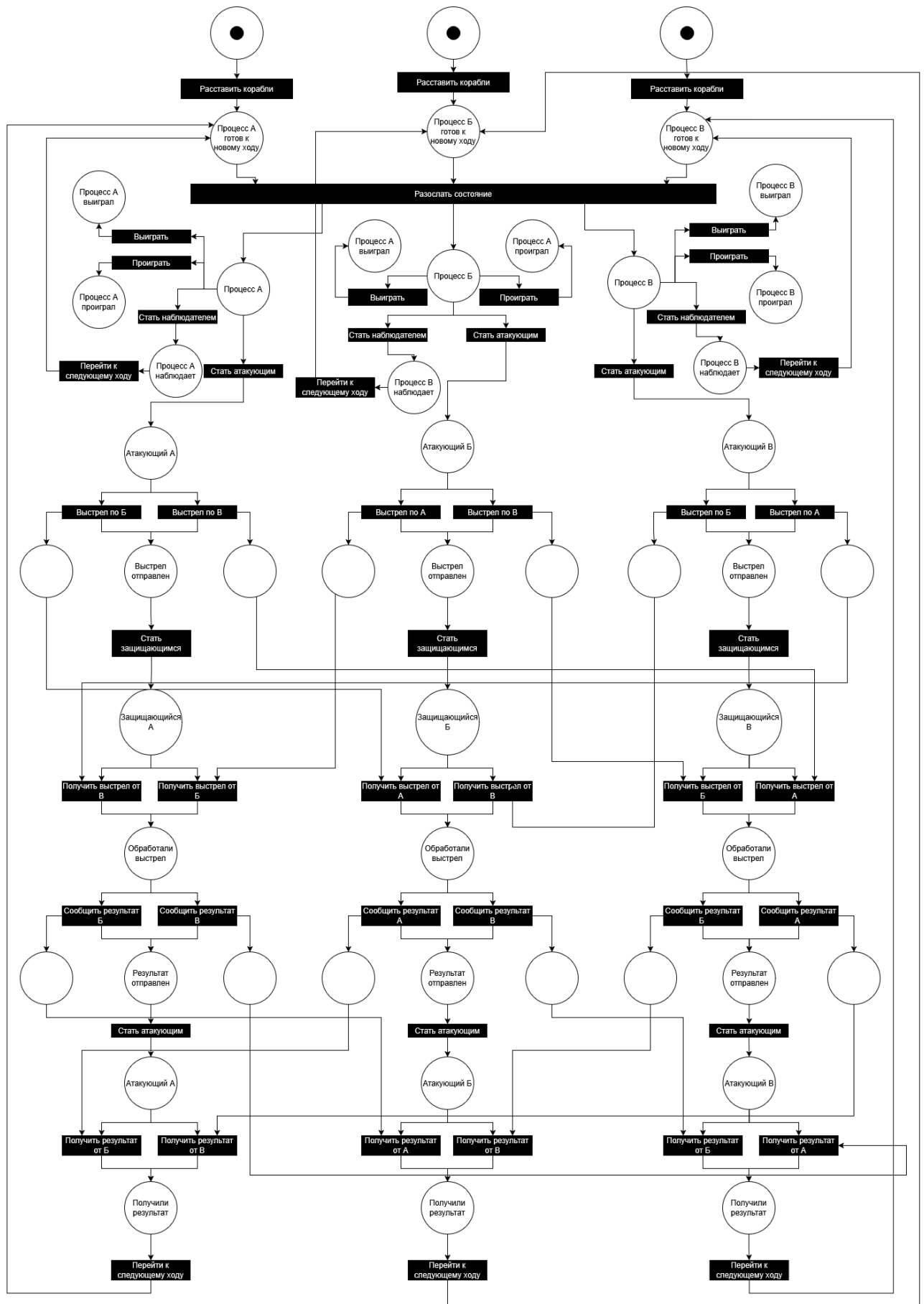


Рисунок 1 – Сеть Петри для Морского Боя на 3х игроков по кольцевому правилу.

Исследуем время выполнения программы при различном размере задачи для различного числа процессов (игроков). Результаты экспериментов представлены в таблице 1. Для каждого числа процессов и размеров поля значения приведены в усреднение на 50 запусков программы (время работы программы считалось по среднему времени завершения всех процессов).

Таблица 1.

Размеры поля	Число процессов										
	2	3	4	5	6	8	10	12	16	24	32
10x10	8.7	16.4	35.0	90.3	144	243	322.8	399.3	558.7	924.8	1065
20x20	45.6	47.9	76.0	132	346	596	666.8	785.2	1375	2688	3470
30x30	82.3	87.8	103	161	446	856	1245	1450	1643	2970	9190
40x40	114	123	134.2	221.9	535.8	1093	1598.6	3029	3992.7	5165.6	13395

По данным из таблицы можно сделать несколько ключевых выводов о поведении параллельной реализации алгоритма морского боя с кольцевой логикой для многопользовательского режима.

Во-первых, время выполнения задачи увеличивается с ростом числа процессов при фиксированном размере поля. Этот рост становится более выраженным для задач большей сложности, то есть для полей большего размера. Как показано на рис. 2, зависимость времени выполнения от количества процессов демонстрирует нелинейный характер, особенно заметный для полей размером 30×30 и 40×40. При анализе зависимости времени выполнения от количества процессов можно заметить четкую границу при 6 процессах. До 6 процессов наблюдается относительно плавный рост времени выполнения, в то время как после 6 процессов происходит значительное ускорение роста сложности. Это хорошо видно на примере поля 40×40: время выполнения возрастает с 535 мс при 6 процессах до 13395 мс при 32 процессах - более чем в 25 раз при росте числа процессов всего в 5 раз. Данный эффект имеет простое архитектурное объяснение: поскольку в используемой системе

имеется 6 физических ядер, работа до 6 процессов обеспечивает оптимальное распределение нагрузки, когда каждый процесс выполняется на отдельном ядре без конкуренции за вычислительные ресурсы. Также резкое увеличение времени выполнения после 6 процессов объясняется комбинацией нескольких факторов, связанных с организацией MPI-коммуникаций. Операция MPI_Allgather, используемая для синхронизации статусов всех игроков, при количестве процессов больше 6 приводит к существенным накладным расходам. Каждый процесс должен обменяться данными со всеми остальными, и при увеличении числа процессов объем коммуникаций растет квадратично. Операции MPI_Send/MPI_Recv также вносят значительный вклад в общее время выполнения. Каждый ход – цепочка сетевых взаимодействий.

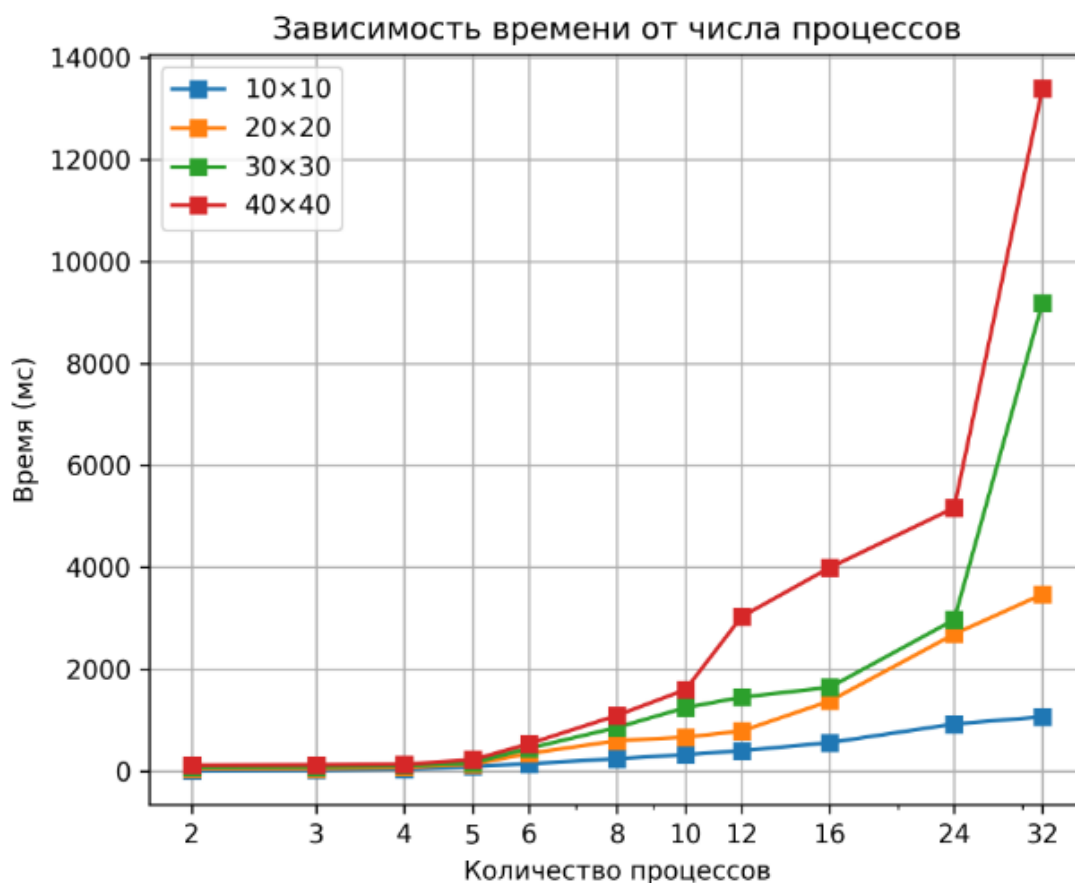


Рисунок 2. Зависимость времени от числа процессов.

Во-вторых, анализ зависимости времени выполнения от размера поля, представленный на рис. 2, показывает практически линейный характер роста. Например, для 4 процессов время выполнения составляет 35.0 мс для поля 10x10, 76.0 мс для 20x20, 103 мс для 30x30 и 134.2 мс для 40x40. Такой

линейный рост, объясняется особенностью алгоритма: количество необходимых ходов для завершения игры растет линейно с размером поля, поскольку плотность кораблей остается постоянной, а стратегия выбора целей является случайной.

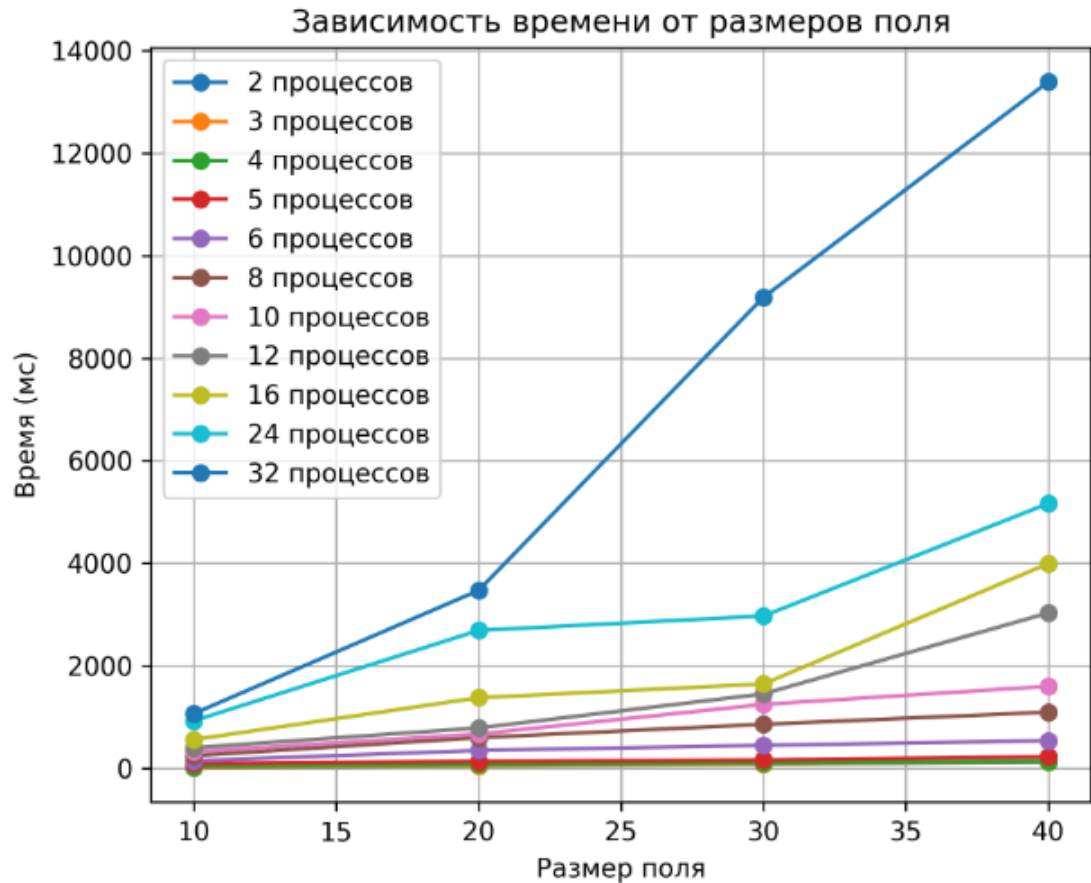


Рисунок 3. Зависимость времени от размеров поля.

Рассчитаем ускорение программы по формуле:

$$S_p(n) = \frac{T_1(n)}{T_p(n)}, \text{ где } p - \text{число процессов, а } n - \text{размеры поля}$$

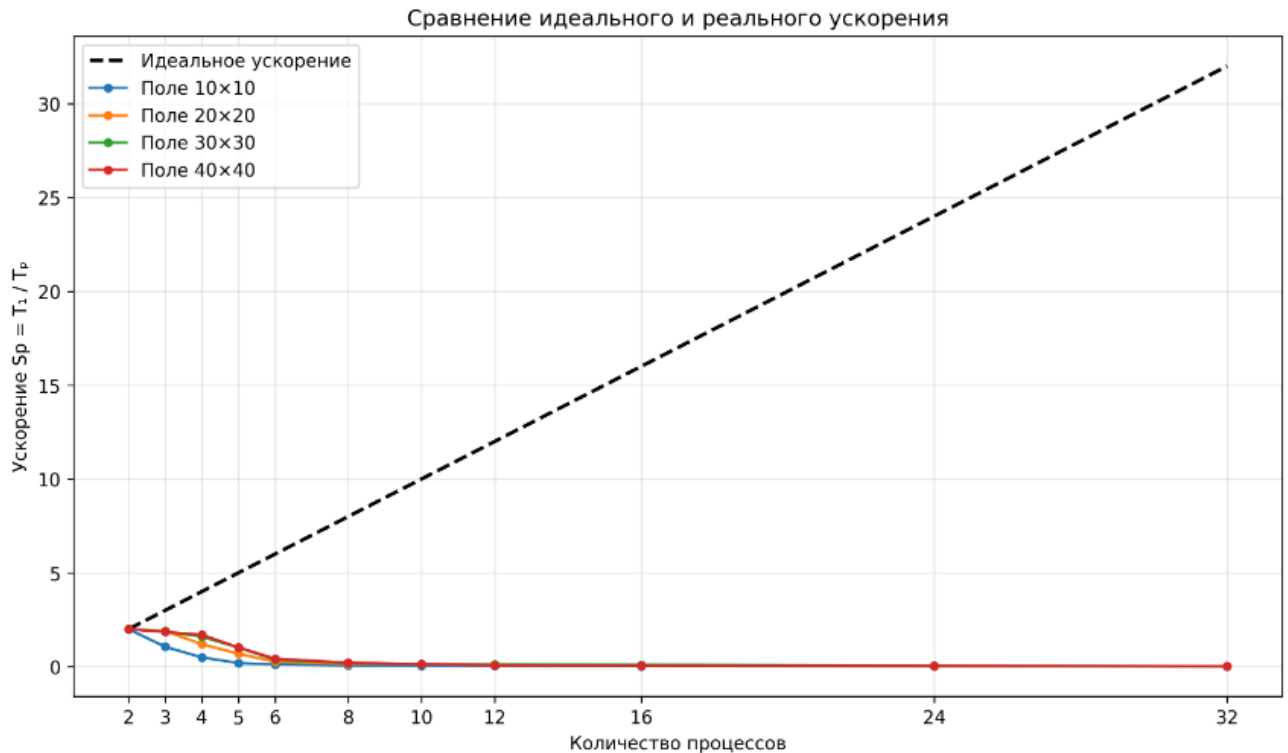


Рисунок 4. Ускорение программы для разных размеров поля при увеличении числа процессов.

Как видно из рисунка 4, характер кривых ускорения для различных размеров игрового поля демонстрирует фундаментальные ограничения параллельной обработки в задачах с интенсивной синхронизацией. Резкое снижение ускорения в диапазоне 2-6 процессов, более выраженное для малых полей, объясняется возрастающей долей накладных расходов на организацию MPI-коммуникаций, в то время как для больших полей снижение плавнее. Разница в скорости деградации обусловлена различным соотношением вычислительной работы и коммуникационных операций: на малых полях игра завершается за меньшее число ходов, тогда как на больших полях число ходов значительно выше, что временно маскирует коммуникационные издержки. Поскольку коммуникационные операции занимают сравнимое время независимо от размера поля, их относительная доля в общем времени выполнения значительно выше для малых полей.

Принципиальное ограничение производительности возникает из-за синхронной природы алгоритма морского боя, где каждый раунд требует

полной синхронизации всех участников. Операция `MPI_Allgather`, выполняемая в каждом раунде для обмена статусами игроков, создает глобальную точку синхронизации, время выполнения которой растет с увеличением количества процессов. После достижения 6 процессов (количества физических ядер в системе) наблюдается качественное изменение характера ускорения - начинается фаза отсутствия ускорения производительности, обусловленная необходимостью разделения ядер между несколькими процессами и резким ростом накладных расходов на переключение контекста.

Морской бой по своей природе является последовательно-параллельной задачей: хотя процессы могут одновременно выполнять вычисления, переход к следующему раунду возможен только после завершения всех операций текущего раунда. Это создает эффект "коммуникационного барьера", где общее время выполнения определяется самым медленным процессом. При большом количестве процессов разброс времени выполнения отдельных операций увеличивается, что приводит к росту времени ожидания и, как следствие, к фактическому отсутствию ускорения несмотря на формальное распараллеливание.

Исходный код программы находится в приложении А, листинг программы для поля 6 на 6 и 3х процессов находится в приложении Б.

Выводы.

В ходе выполнения лабораторной работы были изучены и применены на практике функции обмена данными «точка-точка» в библиотеке `MPI`. Реализована программа многопользовательского морского боя с использованием `MPI_Send`, `MPI_Recv` и `MPI_Allgather` как основного механизма взаимодействия между процессами.

Разработанный алгоритм, основанный на кольцевой топологии процессов, показал свою работоспособность и позволил исследовать характеристики параллельного выполнения. Использование балансирующей формулы для

расчета количества кораблей обеспечило соблюдение стандартной плотности морского боя 20% для полей различных размеров.

Проведенное исследование производительности выявило ключевые закономерности:

- Оптимальный диапазон работы алгоритма составляет 2-6 процессов, что соответствует числу физических ядер в тестовой системе
- Наблюдается четкая граница эффективности при превышении количества процессов над числом доступных ядер
- Большие игровые поля демонстрируют лучшую масштабируемость благодаря более благоприятному соотношению вычислительной работы и коммуникационных издержек

Анализ ускорения показал фундаментальное ограничение для задач с интенсивной синхронизацией - операция `MPI_Allgather`, необходимая для координации состояний игроков, создает глобальную точку синхронизации, время выполнения которой растет с увеличением количества процессов.

ПРИЛОЖЕНИЕ А

ИСХОДНЫЙ КОД ПРОГРАММЫ

```
sea_battle.cpp
#include <iostream>
#include <vector>
#include <algorithm>
#include <mpi.h>
#include <fstream>
#include <numeric>

#define FIELD_SIZE 10
#define DENSITY_PERCENT 20
#define COUNT_SHIPS ((int) (FIELD_SIZE * FIELD_SIZE *
DENSITY_PERCENT) / 100)

enum class PlayerStatus {
    ALIVE = 0,
    DEAD = 1
};

enum class CellState {
    WATER = 0,
    SHIP = 1,
    UNKNOWN = 2
};

enum class ShotResult {
    MISS = 0,
    HIT = 1,
    DEAD = 2
};

struct Cell {
    int x;
    int y;
    CellState state;
};

using Board = std::vector<Cell>;

void safe_cout(int rank, const std::string& message) {
    std::string filename = "debug_player_" + std::to_string(rank) +
".log";
    std::string log_message = "[Игрок " + std::to_string(rank) + "]"
+ message;
    std::ofstream file(filename, std::ios::app);
    file << log_message << std::endl;
    file.close();
}

Board generateBoard(int fieldSize = FIELD_SIZE) {
    Board board;
    board.reserve(fieldSize * fieldSize);
```

```

        for (int y = 0; y < fieldSize; y++) {
            for (int x = 0; x < fieldSize; x++) {
                board.push_back({x, y, CellState::WATER});
            }
        }

        return board;
    };

    void placeShips(Board& board, int fieldSize = FIELD_SIZE, int
countShips = COUNT_SHIPS) {
        int placed = 0;
        std::cout << countShips << std::endl;
        while (placed < countShips) {
            int x = rand() % fieldSize;
            int y = rand() % fieldSize;

            int idx = y * fieldSize + x;

            if (board[idx].state == CellState::WATER) {
                board[idx].state = CellState::SHIP;
                placed++;
            }
        }
    };

    Cell getBoardCell(const Board& board, int x, int y, int fieldSize =
FIELD_SIZE) {
        return board[y * fieldSize + x];
    };

    PlayerStatus getPlayerStatus(const Board& board) {
        bool hasShip = std::any_of(board.begin(), board.end(), [](const
Cell& c){ return c.state == CellState::SHIP; });
        return hasShip ? PlayerStatus::ALIVE : PlayerStatus::DEAD;
    };

    Board generateAvaliableCells(int fieldSize = FIELD_SIZE) {
        Board availableCells;
        availableCells.reserve(fieldSize * fieldSize);

        for (int y = 0; y < fieldSize; y++) {
            for (int x = 0; x < fieldSize; x++) {
                availableCells.push_back({x, y, CellState::UNKNOWN});
            }
        }

        return availableCells;
    };

    Cell getRandomAvaliableCell(Board& availableCells) {
        if (availableCells.empty()) {
            return {-1, -1, CellState::UNKNOWN};
        }

        int idx = rand() % availableCells.size();

```

```

        Cell target = availableCells[idx];

        availableCells[idx] = availableCells.back();
        availableCells.pop_back();

        return target;
    };

    void sendShot(const Cell& target, int destRank) {
        int coords[2] = { target.x, target.y };
        MPI_Send(coords, 2, MPI_INT, destRank, 0, MPI_COMM_WORLD);
    };

    Cell receiveShot(int sourceRank) {
        int coords[2];
        MPI_Recv(coords, 2, MPI_INT, sourceRank, 0, MPI_COMM_WORLD,
MPI_STATUS_IGNORE);

        Cell target = {coords[0], coords[1], CellState::UNKNOWN};

        return target;
    };

    ShotResult processIncomingShot(Board& board, PlayerStatus&
currentStatus, const Cell& shot, int fieldSize = FIELD_SIZE) {
        int idx = shot.y * fieldSize + shot.x;

        if (board[idx].state == CellState::SHIP) {
            board[idx].state = CellState::WATER;
            PlayerStatus newStatus = getPlayerStatus(board);

            if (newStatus == PlayerStatus::DEAD) {
                currentStatus = newStatus;
                return ShotResult::DEAD;
            }
            return ShotResult::HIT;
        }
        return ShotResult::MISS;
    };

    void sendShotResult(ShotResult result, int destRank) {
        int r = static_cast<int>(result);
        MPI_Send(&r, 1, MPI_INT, destRank, 1, MPI_COMM_WORLD);
    };

    ShotResult receiveShotResult(int sourceRank) {
        int r;
        MPI_Recv(&r, 1, MPI_INT, sourceRank, 1, MPI_COMM_WORLD,
MPI_STATUS_IGNORE);
        return static_cast<ShotResult>(r);
    };

    int getAlivePlayersCount(int rank, int size, PlayerStatus myStatus,
std::vector<int>& allStatuses) {
        int myStatusInt = static_cast<int>(myStatus);
        MPI_Allgather(&myStatusInt, 1, MPI_INT, allStatuses.data(), 1,
MPI_INT, MPI_COMM_WORLD);

```

```

        std::string message = "Состояния всех игроков: [";
        for (int i = 0; i < size; i++) {
            message += (allStatuses[i] == 0 ? "ЖИВ" : "МЕРТВ");
            if (i < size - 1) message += ", ";
        }
        message += "];";
        safe_cout(rank, message);

        int aliveCount = 0;
        for (int status : allStatuses) {
            if (status == static_cast<int>(PlayerStatus::ALIVE)) {
                aliveCount++;
            }
        }

        return aliveCount;
    };

    int findNextAlivePlayer(int currentRank, int size, const
std::vector<int>& allStatuses) {
        for (int offset = 1; offset < size; offset++) {
            int nextRank = (currentRank + offset) % size;
            if (allStatuses[nextRank] ==
static_cast<int>(PlayerStatus::ALIVE)) {
                return nextRank;
            }
        }
        return currentRank;
    };

    int findPreviousAlivePlayer(int currentRank, int size, const
std::vector<int>& allStatuses) {
        for (int offset = 1; offset < size; offset++) {
            int prevRank = (currentRank - offset + size) % size;
            if (allStatuses[prevRank] ==
static_cast<int>(PlayerStatus::ALIVE)) {
                return prevRank;
            }
        }
        return currentRank;
    };

    void playGame(int rank, int size) {
        safe_cout(rank, "START GAME");
        int attackTarget = -1;
        int defenseSource = -1;
        int alivePlayersCount = size;

        Board myBoard = generateBoard();
        placeShips(myBoard);
        Board availableCells = generateAvaliableCells();

        PlayerStatus myStatus = PlayerStatus::ALIVE;
        std::vector<int> allStatuses(size);

        while (true) {

```

```

        alivePlayersCount = getAlivePlayersCount(rank, size,
myStatus, allStatuses);
        if (alivePlayersCount <= 1) {
            break;
        }

        int newAttackTarget = findNextAlivePlayer(rank, size,
allStatuses);
        defenseSource = findPreviousAlivePlayer(rank, size,
allStatuses);

        if (attackTarget != newAttackTarget) {
            availableCells = generateAvaliableCells();
        }
        attackTarget = newAttackTarget;

        if (myStatus == PlayerStatus::ALIVE) {
            Cell target = getRandomAvaliableCell(availableCells);
            safe_cout(rank, "Стреляет в игрока: " +
std::to_string(attackTarget) + " по координатам: (" +
std::to_string(target.x) + "," + std::to_string(target.y) + ")");
            sendShot(target, attackTarget);

            Cell incomingShot = receiveShot(defenseSource);
            safe_cout(rank, "Получил выстрел от игрока: " +
std::to_string(defenseSource) + " по координатам: (" +
std::to_string(incomingShot.x) + "," + std::to_string(incomingShot.y) +
")");

            ShotResult myIncomingShotResult =
processIncomingShot(myBoard, myStatus, incomingShot);
            std::string resultStr;
            switch(myIncomingShotResult) {
                case ShotResult::MISS: resultStr = "ПРОМАХ"; break;
                case ShotResult::HIT: resultStr = "ПОПАДАНИЕ";
break;
                case ShotResult::DEAD: resultStr = "УБИТ"; break;
            }
            safe_cout(rank, "Результат выстрела: " + resultStr);
            sendShotResult(myIncomingShotResult, defenseSource);

            ShotResult myShotResult =
receiveShotResult(attackTarget);
            std::string myResultStr;
            switch(myShotResult) {
                case ShotResult::MISS: myResultStr = "ПРОМАХ";
break;
                case ShotResult::HIT: myResultStr = "ПОПАДАНИЕ";
break;
                case ShotResult::DEAD: myResultStr = "УБИТ"; break;
            }
            safe_cout(rank, "Результат атаки: " + myResultStr);
        }
    }

    if (myStatus == PlayerStatus::ALIVE && alivePlayersCount == 1)
{
        safe_cout(rank, "ПОВЕДИЛ!");
    }
}

```

```

        } else if (myStatus == PlayerStatus::ALIVE) {
            safe_cout(rank, "ВЫЖИЛ В НИЧЬЕЙ!");
        } else {
            safe_cout(rank, "ПРОИГРАЛ!");
        }
    };

    int main(int argc, char *argv[]) {
        int rank, size;
        MPI_Status Status;
        MPI_Init(&argc, &argv);
        MPI_Comm_rank(MPI_COMM_WORLD, &rank);
        MPI_Comm_size(MPI_COMM_WORLD, &size);
        std::cout << "Процесс " << rank << ": MPI_Comm_size вернул size
= " << size << std::endl;
        if(size < 2){
            std::cerr << "Запустите минимум 2 процесса для игры!" <<
std::endl;
            MPI_Finalize();
            return 0;
        }

        srand(time(NULL) + rank);

        double startTime = MPI_Wtime();
        playGame(rank, size);
        double endTime = MPI_Wtime();
        double gameTime = (endTime - startTime) * 1000;

        std::vector<double> allTimes;
        if (rank == 0) {
            allTimes.resize(size);
        }

        MPI_Gather(&gameTime, 1, MPI_DOUBLE,
                    allTimes.data(), 1, MPI_DOUBLE,
                    0, MPI_COMM_WORLD);
        MPI_Finalize();

        if (rank == 0) {
            double maxTime = *std::max_element(allTimes.begin(),
allTimes.end());
            double avgTime = std::accumulate(allTimes.begin(),
allTimes.end(), 0.0) / size;

            std::cout << "\n=== РЕЗУЛЬТАТЫ ДЛЯ " << size << " ПРОЦЕССОВ
===" << std::endl;
            std::cout << "Максимальное время: " << maxTime << " мс" <<
std::endl;
            std::cout << "Среднее время: " << avgTime << " мс" <<
std::endl;

            // Записываем в файл для последующего анализа
            std::ofstream file("timing_results" +
std::to_string(FIELD_SIZE) + ".txt", std::ios::app);
            file << size << " " << maxTime << " " << avgTime <<
std::endl;

```



```
        file.close();
    }

    std::cout << "Процесс " << rank
               << " завершил игру за "
               << gameTime << " мс" << std::endl;
    return 0;
};
```

ПРИЛОЖЕНИЕ Б

ЛИСТИНГ ПРОГРАММЫ

Process_1_log.txt

[illegible]

[illegible]

[illegible]

```
[Игрок 1] START GAME
[Игрок 1] Состояния всех игроков: [ЖИВ, ЖИВ, ЖИВ]
[Игрок 1] Стреляет в игрока: 2 по координатам: (2,3)
[Игрок 1] Получил выстрел от игрока: 0 по координатам: (1,3)
[Игрок 1] Результат выстрела: ПРОМАХ
[Игрок 1] Результат атаки: ПРОМАХ
[Игрок 1] Состояния всех игроков: [ЖИВ, ЖИВ, ЖИВ]
[Игрок 1] Стреляет в игрока: 2 по координатам: (0,4)
[Игрок 1] Получил выстрел от игрока: 0 по координатам: (4,3)
```

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[Игрок 2] Состояния всех игроков: [МЕРТВ, ЖИВ, ЖИВ]
[Игрок 2] Стреляет в игрока: 1 по координатам: (2,1)
[Игрок 2] Получил выстрел от игрока: 1 по координатам: (2,0)
[Игрок 2] Результат выстрела: УБИТ
[Игрок 2] Результат атаки: ПРОМАХ
[Игрок 2] Состояния всех игроков: [МЕРТВ, ЖИВ, МЕРТВ]
[Игрок 2] ПРОИГРАЛ!